

EXPERIMENT – 2

Stages of Compilation and Execution of a C Program

1. CODE :-

```
#include <stdio.h>

int main(void)
{
    float radius, area;

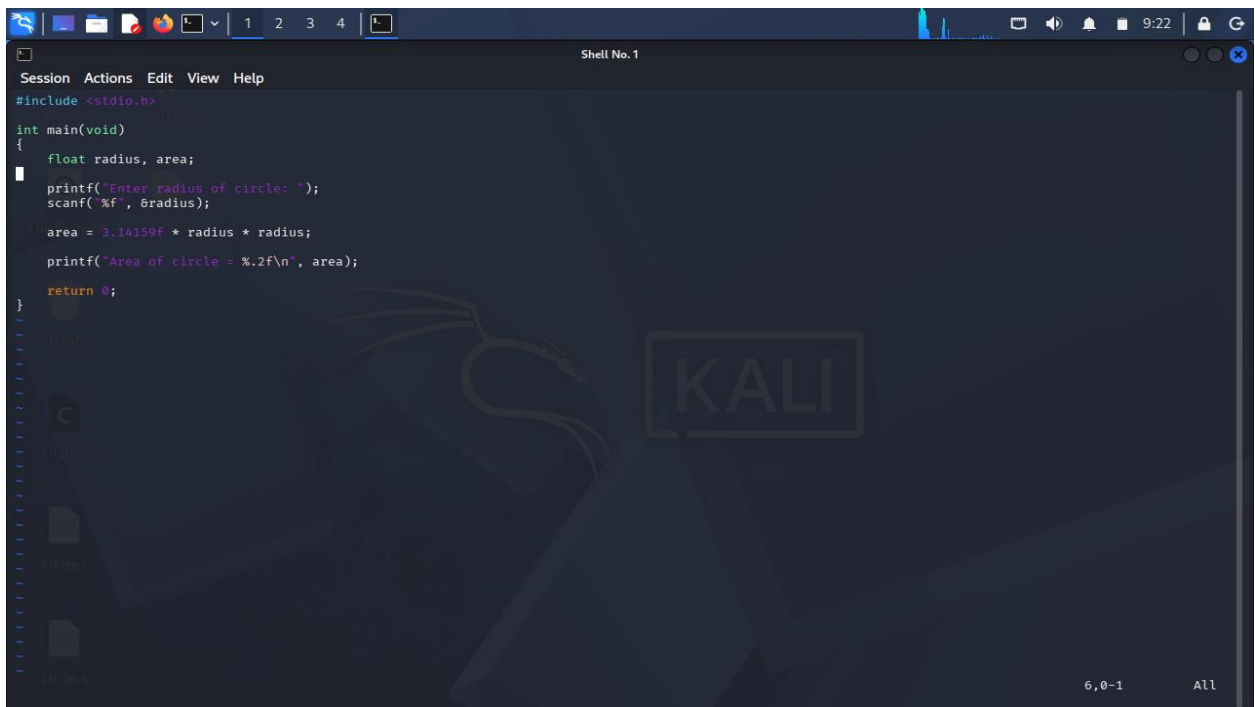
    printf("Enter radius of circle: ");

    scanf("%f", &radius);

    area = 3.14159f * radius * radius;

    printf("Area of circle = %.2f\n", area);

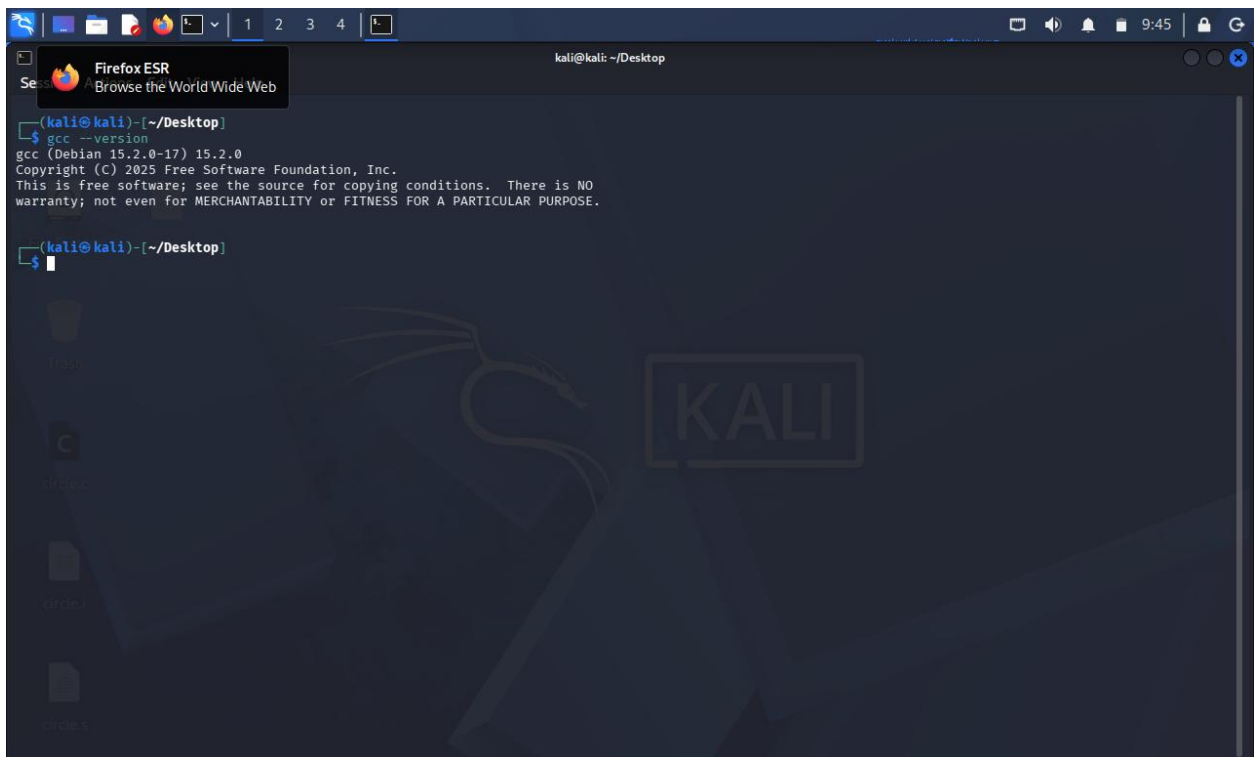
    return 0;
}
```



2. GCC VERSION :-

GCC (GNU Compiler Collection) is used to compile the C program. The installed GCC version can be checked before starting the compilation process.

COMMAND :- gcc --version

A screenshot of a Kali Linux desktop environment. A terminal window is open, displaying the command 'gcc --version' and its output. The output shows 'gcc (Debian 15.2.0-17) 15.2.0' and copyright information for the Free Software Foundation, Inc. The desktop background features a Kali Linux logo and the word 'KALI' in a large, stylized font. A Firefox ESR browser window is also visible in the background.

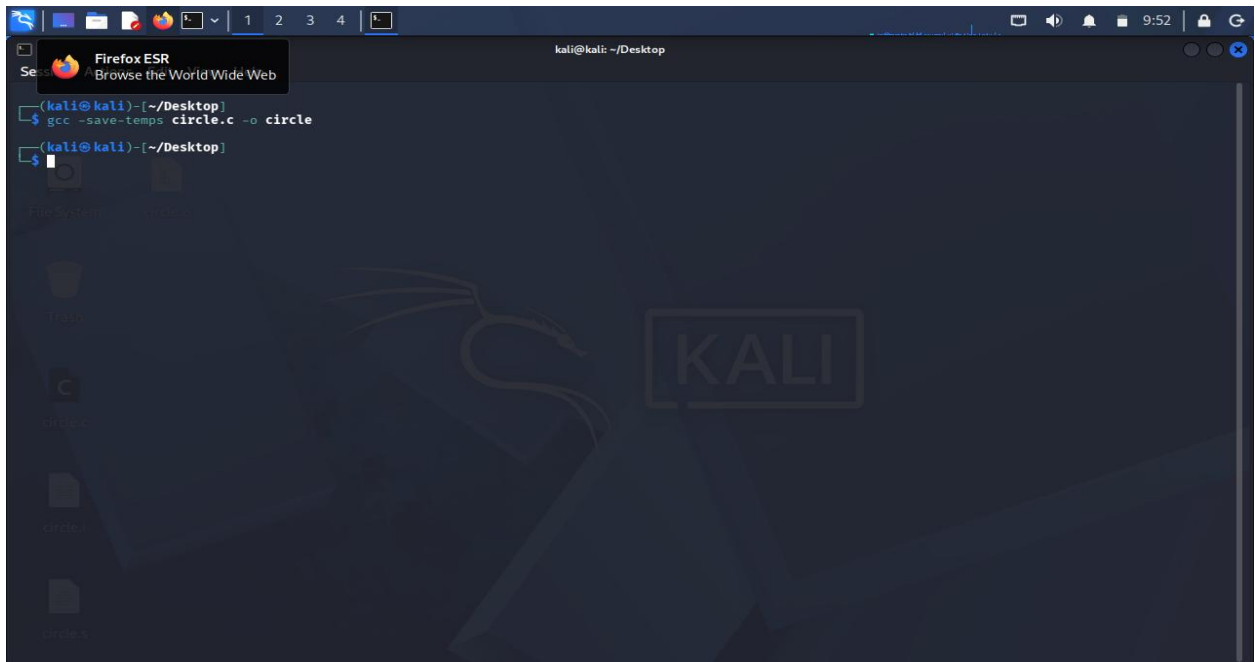
```
(kali@kali)-[~/Desktop]
$ gcc --version
gcc (Debian 15.2.0-17) 15.2.0
Copyright (C) 2025 Free Software Foundation, Inc.
This is free software; see the source for copying conditions. There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.

(kali@kali)-[~/Desktop]
$
```

3. COMPILE USING -save-temps :-

The -save-temps option tells GCC to keep the intermediate files generated during compilation. This allows us to observe the different stages of the compilation process.

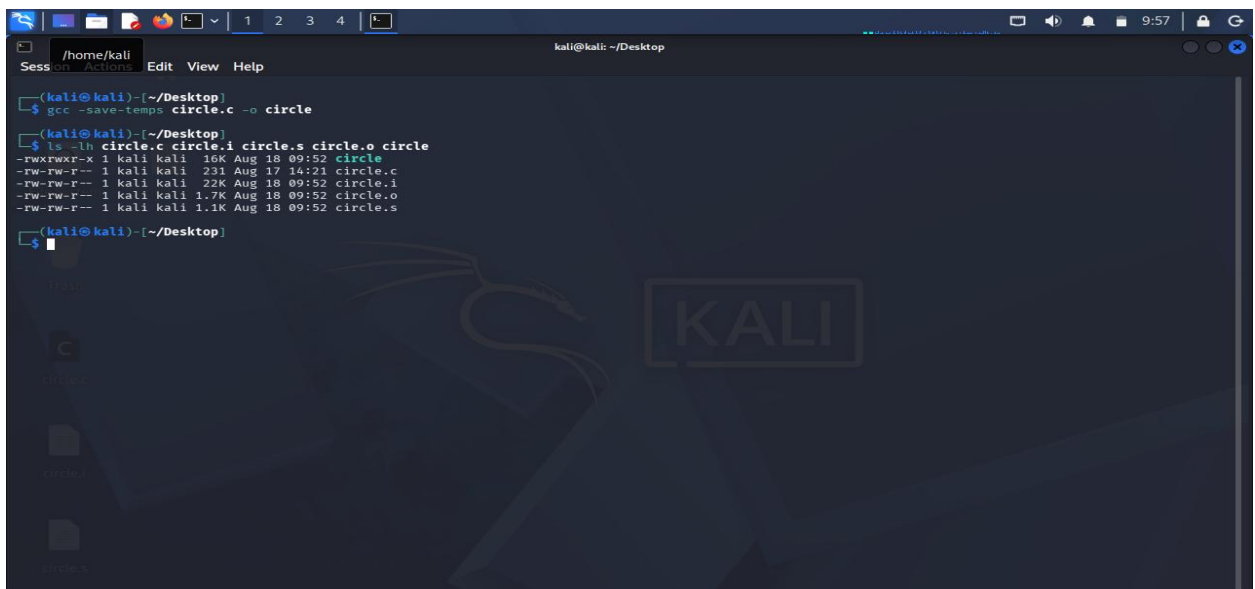
COMMAND :- gcc -save-temps circle.c -o circle



4. GENERATED IMMEDIATE FILES :-

After compilation, GCC generates the intermediate files along with the final executable. The .i, .s, and .o files represent different stages of compilation.

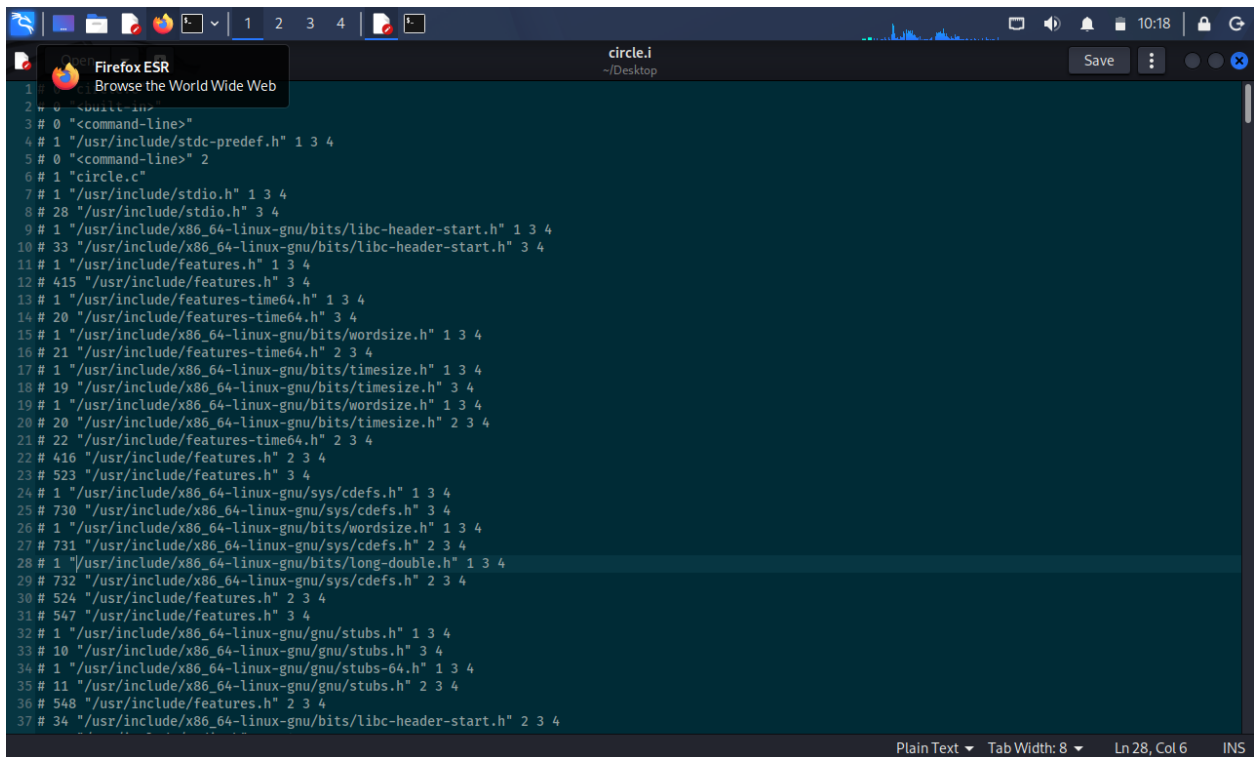
COMMAND :- `ls -lh circle.c circle.i circle.s circle.o circle`



5. PREPROCESSING STAGE – circle.i :-

During preprocessing, directives such as `#include <stdio.h>` are processed and expanded. The resulting preprocessed source code is stored in `circle.i`.

COMMAND :- `gedit circle.i`



```
1 2
3 # 0 "<command-line>"
4 # 1 "/usr/include/stdc-predef.h" 1 3 4
5 # 0 "<command-line>" 2
6 # 1 "circle.c"
7 # 1 "/usr/include/stdio.h" 1 3 4
8 # 28 "/usr/include/stdio.h" 3 4
9 # 1 "/usr/include/x86_64-linux-gnu/bits/libc-header-start.h" 1 3 4
10 # 33 "/usr/include/x86_64-linux-gnu/bits/libc-header-start.h" 3 4
11 # 1 "/usr/include/features.h" 1 3 4
12 # 415 "/usr/include/features.h" 3 4
13 # 1 "/usr/include/features-time64.h" 1 3 4
14 # 20 "/usr/include/features-time64.h" 3 4
15 # 1 "/usr/include/x86_64-linux-gnu/bits/wordsize.h" 1 3 4
16 # 21 "/usr/include/features-time64.h" 2 3 4
17 # 1 "/usr/include/x86_64-linux-gnu/bits/timesize.h" 1 3 4
18 # 19 "/usr/include/x86_64-linux-gnu/bits/timesize.h" 3 4
19 # 1 "/usr/include/x86_64-linux-gnu/bits/wordsize.h" 1 3 4
20 # 20 "/usr/include/x86_64-linux-gnu/bits/timesize.h" 2 3 4
21 # 22 "/usr/include/features-time64.h" 2 3 4
22 # 416 "/usr/include/features.h" 2 3 4
23 # 523 "/usr/include/features.h" 3 4
24 # 1 "/usr/include/x86_64-linux-gnu/sys/cdefs.h" 1 3 4
25 # 730 "/usr/include/x86_64-linux-gnu/sys/cdefs.h" 3 4
26 # 1 "/usr/include/x86_64-linux-gnu/bits/wordsize.h" 1 3 4
27 # 731 "/usr/include/x86_64-linux-gnu/sys/cdefs.h" 2 3 4
28 # 1 "/usr/include/x86_64-linux-gnu/bits/long-double.h" 1 3 4
29 # 732 "/usr/include/x86_64-linux-gnu/sys/cdefs.h" 2 3 4
30 # 524 "/usr/include/features.h" 2 3 4
31 # 547 "/usr/include/features.h" 3 4
32 # 1 "/usr/include/x86_64-linux-gnu/gnu/stubs.h" 1 3 4
33 # 10 "/usr/include/x86_64-linux-gnu/gnu/stubs.h" 3 4
34 # 1 "/usr/include/x86_64-linux-gnu/gnu/stubs-64.h" 1 3 4
35 # 11 "/usr/include/x86_64-linux-gnu/gnu/stubs.h" 2 3 4
36 # 548 "/usr/include/features.h" 2 3 4
37 # 34 "/usr/include/x86_64-linux-gnu/bits/libc-header-start.h" 2 3 4
```

6. COMPILATION STAGE – circle.s :-

The compiler converts the preprocessed C code into assembly language. The assembly-language output is stored in the `circle.s` file.

COMMAND :- `gedit circle.s`

```
Firefox ESR
Browse the World Wide Web

circle.s
~/Desktop

1 |
2 |
3 |
4 | .LC0:
5 |   .string "Enter radius of circle: "
6 | .LC1:
7 |   .string "%f"
8 | .LC3:
9 |   .string "Area of circle = %.2f\n"
10 |
11 | .text
12 | .globl main
13 | .type main, @function
14 |
15 | .LFB0:
16 |   .cfi_startproc
17 |   pushq %rbp
18 |   .cfi_def_cfa_offset 16
19 |   .cfi_offset 6, -16
20 |   movq %rsp, %rbp
21 |   .cfi_def_cfa_register 6
22 |   subq $16, %rsp
23 |   leaq .LC0(%rip), %rax
24 |   movq %rax, %rdi
25 |   movl $0, %eax
26 |   call printf@PLT
27 |   leaq -8(%rbp), %rax
28 |   leaq .LC1(%rip), %rdx
29 |   movq %rax, %rsi
30 |   movq %rdx, %rdi
31 |   movl $0, %eax
32 |   call __isoc23_scanf@PLT
33 |   movss -8(%rbp), %xmm1
34 |   movss .LC2(%rip), %xmm0
35 |   mulss %xmm0, %xmm1
36 |   movss -8(%rbp), %xmm0
37 |   mulss %xmm1, %xmm0
38 |   movss %xmm0, -4(%rbp)
39 |
40 |
41 |
42 |
43 |
44 |
45 |
46 |
47 |
48 |
49 |
50 |
51 |
52 |
53 |
54 |
55 |
56 |
57 |
58 |
59 |
60 |
61 |
62 |
63 |
64 |
65 |
66 |
67 |
68 |
69 |
70 |
71 |
72 |
73 |
74 |
75 |
76 |
77 |
78 |
79 |
80 |
81 |
82 |
83 |
84 |
85 |
86 |
87 |
88 |
89 |
90 |
91 |
92 |
93 |
94 |
95 |
96 |
97 |
98 |
99 |
100 |
```

7. ASSEMBLY STAGE – circle.s :-

The assembler converts the assembly instructions into machine-level object code. GCC saves this output as the circle.o object file.

COMMAND :- (i) objdump -s circle.o

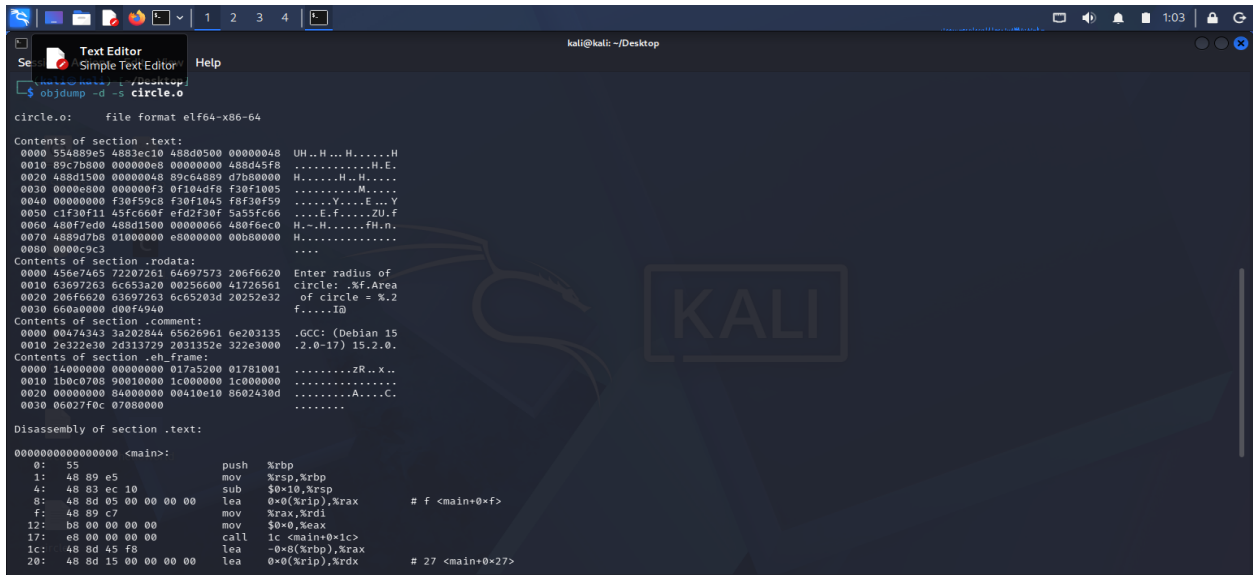
```
kali@kali: ~/Desktop
$ objdump -s circle.o

circle.o: file format elf64-x86-64

Disassembly of section .text:

0000000000000000 <main>:
0: 55          push %rbp
1: 48 89 e5    mov %rsp,%rbp
4: 48 83 ec 10 sub $0x10,%rsp
8: 48 8d 05 00 00 00 00 lea 0x0(%rip),%rax # f <main+0xf>
f: 48 89 c7    mov %rax,%rdi
12: b8 00 00 00 00 mov $0x0,%eax
17: e8 00 00 00 00 call 1c <main+0x1c>
1c: 48 8d 45 f8 lea -0x8(%rbp),%rax
20: 48 8d 15 00 00 00 00 lea 0x0(%rip),%rdx # 27 <main+0x27>
27: 48 89 c6    mov %rax,%rsi
2a: 48 89 d7    mov %rdx,%rdi
2d: b8 00 00 00 00 mov $0x0,%eax
32: e8 00 00 00 00 call 37 <main+0x37>
37: f3 0f 10 4d f8 movss -0x8(%rbp),%xmm1
3c: f3 0f 10 05 00 00 00 movss 0x0(%rip),%xmm0 # 44 <main+0x44>
43: 00
44: f3 0f 59 c8 mulss %xmm0,%xmm1
48: f3 0f 10 45 f8 movss -0x8(%rbp),%xmm0
4d: f3 0f 59 c1 mulss %xmm1,%xmm0
51: f3 0f 11 45 fc movss %xmm0,-0x4(%rbp)
56: 66 0f ef d2 pxor %xmm2,%xmm2
5a: f3 0f 5a 55 fc cvtsq2sd -0x4(%rbp),%xmm2
5f: 66 48 0f 7e d0 movq %xmm2,%rax
64: 48 8d 15 00 00 00 00 lea 0x0(%rip),%rdx # 6b <main+0x6b>
6b: 66 48 0f 6e c0 movq %rax,%xmm0
70: 48 89 d7    mov %rdx,%rdi
73: b8 01 00 00 00 mov $0x1,%eax
78: e8 00 00 00 00 call 7d <main+0x7d>
7d: b8 00 00 00 00 mov $0x0,%eax
82: c9         leave
83: c3         ret
```

(ii)Objdump -s -d circle.o



```
kali@kali: ~/Desktop
$ objdump -d -s circle.o

circle.o:      file format elf64-x86-64

Contents of section .text:
0000 554889e5 4883ec10 488d0500 00000048  UH..H...H.....H
0010 89c7b800 000000e8 00000000 488d45f8  .....H.E.
0020 488d1500 00000048 89c64889 47b80000  H.....H..H....
0030 0000e800 000000f3 0f104df8 f30f1005  .....M....
0040 00000000 f30f59c8 f30f1045 f8f30f59  .....Y....E...Y
0050 c1f30f11 45fc668f efd2f38f 5a55fc66  ...E.f.....ZU.f
0060 488f7edd 488d1500 00000048 488f6ec0  H..H.....fH.n.
0070 4889d7b8 01000000 e8000000 00b80000  H.....
0080 0000c9c3  ....
0090 00000000 00000000 00000000 00000000  ....

Contents of section .rodata:
0000 456e7465 72207261 64897573 206f6620  Enter radius of
0010 63697263 6c653a20 00256600 41726561  circle: %.f.Area
0020 206f6620 63697263 6c65203d 20252e32  of circle = %.2
0030 666a0000 d00f4940  ....f.....l0

Contents of section .comment:
0000 00474343 3a202844 65626961 6e203135  .GCC: (Debian 15
0010 2e322e30 2d313729 2031352e 322e3000  .2.0-17) 15.2.0.

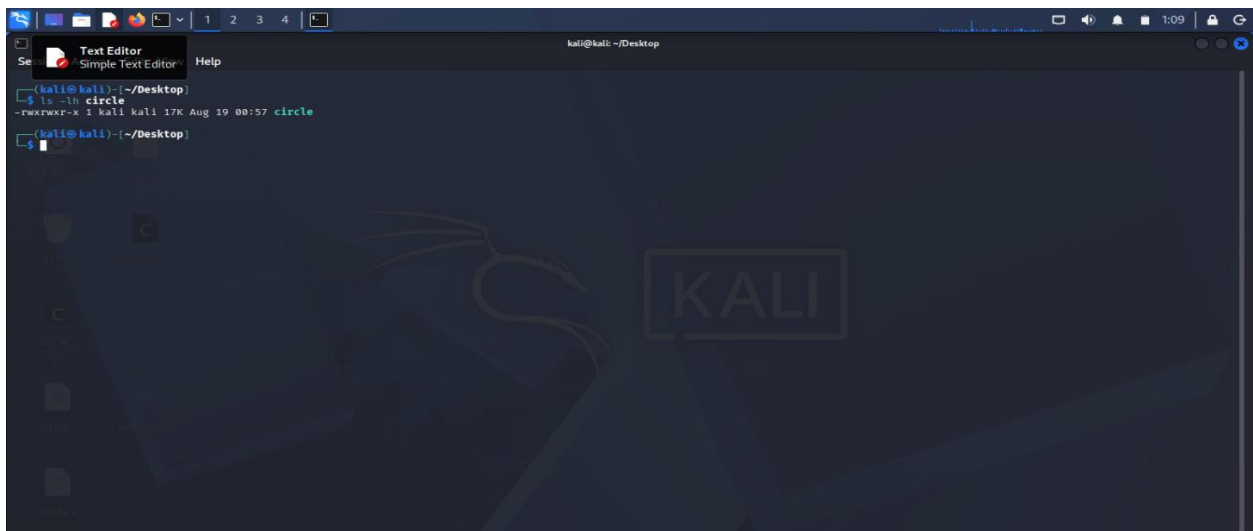
Contents of section .eh_frame:
0000 14000000 00000000 017a5200 01781001  .....zR...X...
0010 1b0c0708 90010000 1c000000 1c000000  .....
0020 00000000 84000000 00410e10 8602430d  .....A...C.
0030 06027f0c 07080000  ....

Disassembly of section .text:
0000000000000000 <main>:
0: 55                push    %rbp
1: 48 89 e5          mov     %rsp,%rbp
4: 48 83 ec 10       sub     $0x10,%rsp
8: 48 8d 05 00 00 00 lea     0x0(%rip),%rax    # f <main+0xf>
f: 48 89 c7          mov     %rax,%rdi
12: b8 00 00 00 00    mov     $0x0,%eax
17: e8 00 00 00 00    call   1c <main+0x1c>
1c: 48 8d 45 f8       lea     -0x8(%rbp),%rax
20: 48 8d 15 00 00 00 lea     0x0(%rip),%rdx    # 27 <main+0x27>
```

8. LINKING STAGE – circle :-

The linker combines the object code with the required libraries and creates the final executable file. In this experiment, the executable is named circle.

COMMAND :- `ls -lh circle`



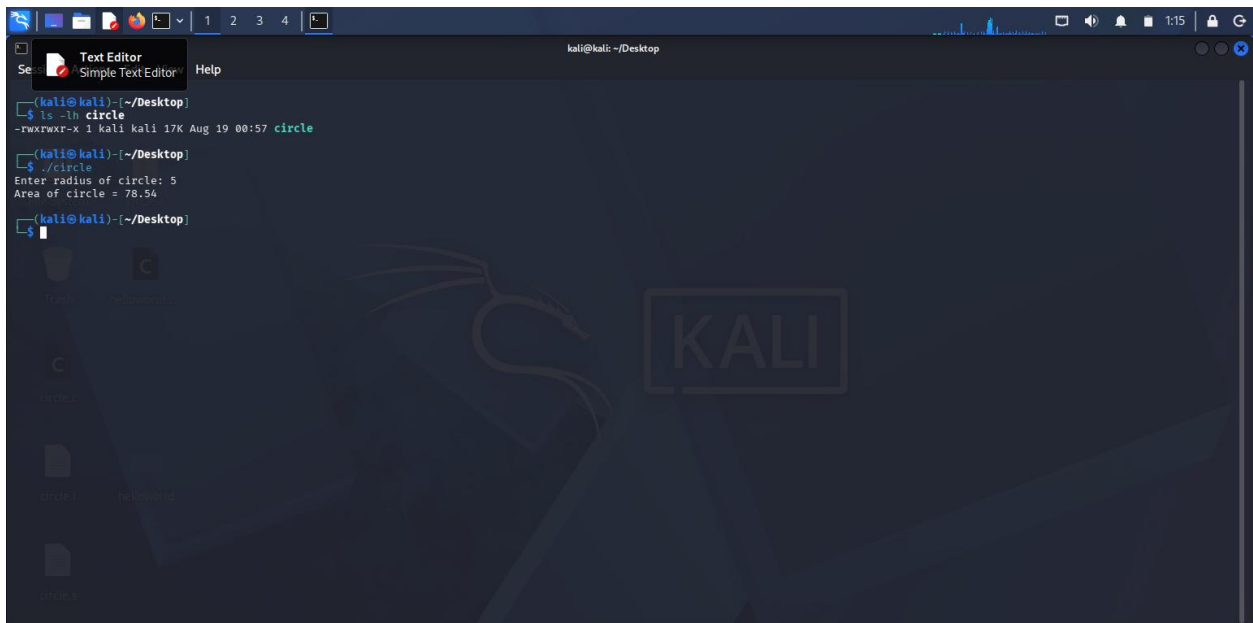
```
kali@kali: ~/Desktop
$ ls -lh circle
-rwxrwxr-x 1 kali kali 17K Aug 19 00:57 circle

kali@kali: ~/Desktop
```

9. EXECUTION OF THE PROGRAM :-

The final executable is run from the terminal. The program accepts the radius of the circle and calculates its area using the formula $\text{Area} = \pi r^2$.

COMMAND :- `./circle`



```
(kali@kali)-[~/Desktop]
└─$ ls -lh circle
-rwxrwxr-x 1 kali kali 17K Aug 19 00:57 circle
└─$ ./circle
Enter radius of circle: 5
Area of circle = 78.54
└─$
```

10. COMPILATION PROCESS SUMMARY :-

`circle.c`



Preprocessing



`circle.i`



Compilation



circle.s



Assembly



circle.o



Linking



circle



Execution



Output: Area of Circle